

1) Smart Electricity Billing System :

i) Declaration :

int units ;

float rate, tax_rate ;

float energy_cost, tax_amount, total_bill ;

ii) Three-Address Code :

 $t1 = \text{units} * \text{rate}$ $t2 = t1 * \text{tax_rate}$ $t3 = t1 + t2$ $\text{energy_cost} = t1$ $\text{tax_amount} = t2$ $\text{total_bill} = t3$

iii) Quadruples :

Op	Arg 1	Arg 2	Result
*	units	rate	t1
*	t1	tax_rate	t2
+	t1	t2	t3
=	t1	-	energy_cost
=	t2	-	tax_amount
=	t3	-	total_bill

Triples :

0 : (*, units, rate)

1 : (*, (0), tax_rate)

2 : (+, (0), (1))

3 : (=, (0), energy_cost)

4 : (=, (1), tax_amount)

5 : (=, (2), total_bill)

Indirect Triples:

Pointer	Triple
P0	0
P1	1
P2	2
P3	3
P4	4
P5	5

iv) Optimization:

t_1 is a common subexpression because energy-cost already contains $\text{units} \times \text{rate}$.

v) Optimized code:

$t_1 = \text{units} * \text{rate}$

$t_2 = t_1 * \text{tax_rate}$

$\text{total_bill} = t_1 + t_2$

2) Employee Promotion Evaluation:

i) Boolean Expression:

$\text{experience} \geq 5 \text{ AND}$

$(\text{performance} > 90 \text{ OR } \text{certification} == \text{TRUE})$

ii) Short-Circuit code:

if $\text{experience} < 5$ goto Lfail

if $\text{performance} > 90$ goto Lpromote

if $\text{certification} == \text{TRUE}$ goto Lpromote

goto Lfail

Lpromote : $\text{promotion} = \text{TRUE}$

goto Lend

Lfail : $\text{promotion} = \text{FALSE}$

Lend :

Backpatching:

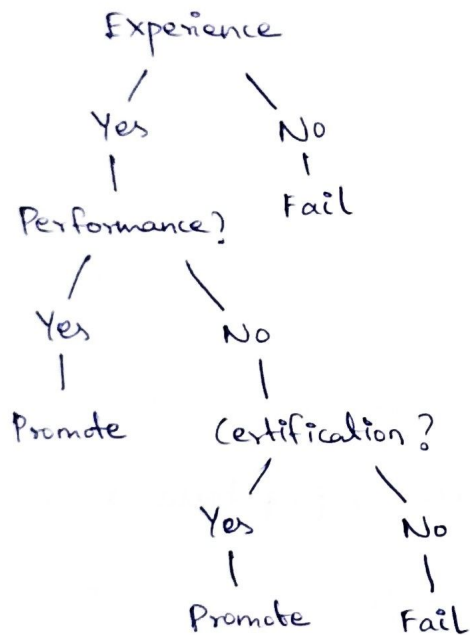
if experience < 5 goto Lfail

if performance > 90 goto Lpromote

if certification == TRUE goto Lpromote

iv) Final code: Same labelled code above.

v) CFG:



3) Hospital Patient Priority System:

i) Conditional TAC:

if priority == 1 goto L1

if priority == 2 goto L2

if priority == 3 goto L3

if priority == 4 goto L4

goto L5

L1: CRITICAL

L2: HIGH

L3: MEDIUM

L4: LOW

L5: Invalid

ii) Jump Table:

goto JT[priority]

JT[1] → CRITICAL

JT[2] → HIGH

JT[3] → MEDIUM

JT[4] → LOW

iii) Quadruples:

(=, priority, 1, L1)

(=, priority, 2, L2)

(=, priority, 3, L3)

(=, priority, 4, L4)

iv) Comparison:

Conditional branching needs multiple comparisons; jump table gives faster direct access.

v) Recommendation:

Jump table, because quick priority selection is important in hospitals.

4) E-Commerce Payment Processing:

i) Intermediate code:

Param cart

t1 = call calculateAmount, 1

param t1

call processPayment, 1

param t1

call sendConfirmation, 1

ii) Parameter Passing:

cart → calculateAmount()

t1 → processPayment()

t1 → sendConfirmation()

Activation Record :

Return Address

Parameters

Local variables

Temporaries

Control Link

iv) Stack :

main()



calculate Amount (cart)



process Payment (t1)



send Confirmation (t1)

v) Procedure Management :

Pass parameters → create activation record → call procedure → execute → return → remove activation record.

5) Smart ATM Withdrawal Verification :

i) Boolean Expression :

balance $\geq$ amount AND

(PIN == TRUE OR biometric == TRUE)

ii) Short-Circuit Code :

if balance < amount goto Lfail

if PIN == TRUE goto Lsuccess

if biometric == TRUE goto Lsuccess
goto Lfail

Lsuccess : withdrawal = allowed

goto Lend

Lfail : withdrawal = denied

Lend :

iii) Backpatching:

if balance < amount goto Lfail

if PIN == TRUE goto Lsuccess

if biometric == TRUE goto Lsuccess

iv) Final Code:

Same labelled code above.

v) CFG:

